COMSATS UNIVERSITY ISLAMABAD

SAHIWAL CAMPUS

PROJECT REPORT

Event Registration System

A Web Application Built with Python Flask and SQLite

Submitted By	Hasnain Ali
Course	Web / Backend Development Project
Instructor	
Submission Date	____10-10-2026_____

Table of Contents

- 1. Executive Summary
- 2. Project Introduction
- 3. Project Objectives
- 4. Scope and Requirements
- 5. Tools and Technologies
- 6. System Design and Architecture
- 7. Project Structure
- 8. Main Features
- 9. API Documentation
- 10. Database Design
- 11. How to Run the Project
- 12. Testing and Verification
- 13. Screenshots and Evidence
- 14. Limitations and Future Improvements
- 15. Conclusion
- 16. References

1. Executive Summary

The Event Registration System is a web application designed to make event creation and participant registration easier. An organizer can add events, visitors can register with their name and email, and the organizer can view participants for each event. Python and Flask provide the backend, SQLite stores records locally, and HTML, CSS, and JavaScript provide a modern dashboard with event cards, summary statistics, forms, and modal windows.

2. Project Introduction

Managing events manually can make it difficult to keep event details and participant records organized. This project provides a simple digital solution. The interface communicates with Flask routes, and event and registration data are stored in database tables so that records remain available after page refreshes and application restarts.

3. Project Objectives

- Create a clear and professional interface for event management.
- Allow organizers to add events with a name, date, location, and description.
- Allow participants to register for an existing event with a name and email address.
- Store event and registration information in an SQLite database.
- Display saved events and the participants registered for each event.
- Provide API endpoints that can be tested and extended later.

4. Scope and Requirements

4.1 Functional Requirements

- Create Event: validate required fields and save a new event.
- List Events: retrieve and display all saved events.
- Event Details: retrieve one event using its ID.
- Register Participant: add a participant to an existing event.
- View Registrations: list participants for a selected event.
- Input Validation: reject missing required fields and unknown event IDs.

4.2 Non-Functional Requirements

- Usability: controls and forms should be understandable to new users.
- Maintainability: templates and backend logic are kept in separate files.
- Data Persistence: records are stored in SQLite.
- Responsive Design: the interface adjusts to different screen sizes.
- Basic Reliability: API routes return JSON responses and relevant HTTP status codes.

5. Tools and Technologies

Technology	Purpose	Use in This Project

Python	Programming language	Backend logic and database operations
Flask	Web framework	Routes, requests, responses, and template rendering
SQLite	Database	Stores events and participant registrations
HTML	Page structure	Forms, navigation, cards, and modal windows
CSS	Visual styling	Colors, spacing, responsive layout, and effects
JavaScript	Browser interaction	Fetch requests and dynamic page updates
VS Code	Development environment	Editing and running the project

6. System Design and Architecture

The application follows a client-server structure. The browser displays the HTML, CSS, and JavaScript interface. JavaScript sends HTTP requests to Flask endpoints. Flask validates incoming data, runs parameterized SQL statements, and reads or writes records in SQLite. The server returns JSON responses that the interface uses to update the page.

- Presentation Layer: templates/index.html contains HTML, CSS, and JavaScript.
- Application Layer: app.py defines routes and handles validation.
- Data Layer: events.db stores events and registrations.

7. Project Structure

```
Event_Registration_System/
├── app.py
├── requirements.txt
├── events.db  (created automatically when the app starts)
├── venv/  (Python virtual environment)
└── templates/
    └── index.html
```

The events.db file is generated automatically when init_db() runs.

8. Main Features

8.1 Professional Dashboard

The dashboard includes a navigation bar, gradient welcome banner, summary statistics, event cards, and a create-event panel. CSS media queries help the layout adapt to smaller screens.

8.2 Event Creation

The event form accepts a name, date, location, and optional description. JavaScript sends the values to Flask as JSON. The backend checks required fields and saves the event in the events table.

8.3 Participant Registration

A visitor selects an event and submits a full name and email address. The request includes the event ID. The backend verifies that the event exists before saving the participant in the registrations table.

8.4 Participant List

The Guests button requests the registrations for a particular event and displays the returned names and email addresses in a modal window.

8.5 Summary Statistics

The dashboard displays the total number of saved events and registrations. The figures are retrieved from the backend and updated when the event list is refreshed.

9. API Documentation

Method and Endpoint	Purpose	Expected Result
GET /	Open dashboard	HTML page
GET /events	List all events	JSON array
POST /events	Create an event	201 on success; 400 if required fields are missing
GET /events/<event_id>	Get one event	JSON object or 404
POST /events/<event_id>/register	Register participant	201 on success; 400 or 404 on error
GET /events/<event_id>/registrations	List participants	JSON array or 404

9.1 Example Event Creation Request

POST /events
Content-Type: application/json

```
{
 "name": "Web Development Workshop",
 "date": "2026-11-15",
 "location": "Main Auditorium",
 "description": "A beginner-friendly web development session"
}
```

A successful request returns a JSON message and the ID assigned to the new event.

9.2 Example Registration Request

POST /events/1/register
Content-Type: application/json

```
{
 "participant_name": "Ali Raza",
 "email": "ali@example.com"
}
```

The event ID must refer to an existing event. The backend returns an error if the event does not exist or required registration fields are missing.

10. Database Design

10.1 Events Table

Field	Type	Description
id	INTEGER	Primary key, automatically generated
name	TEXT	Required event name
date	TEXT	Required event date
location	TEXT	Required event location
description	TEXT	Optional event description

10.2 Registrations Table

Field	Type	Description
id	INTEGER	Primary key, automatically generated
event_id	INTEGER	ID of the event being registered for
participant_name	TEXT	Required participant name
email	TEXT	Required participant email

Each registration is associated with an event through event_id. The current implementation checks that the event exists before inserting a registration. Duplicate-registration prevention and email verification are not implemented.

11. How to Run the Project

1. Open the Event_Registration_System folder in Visual Studio Code.
2. Open Terminal → New Terminal.
3. Activate the virtual environment: venv\Scripts\activate
4. Install dependencies if required: pip install -r requirements.txt
5. Start the server: python app.py
6. Open http://127.0.0.1:5000 in a browser.
7. Keep the terminal open while testing the website.

12. Testing and Verification

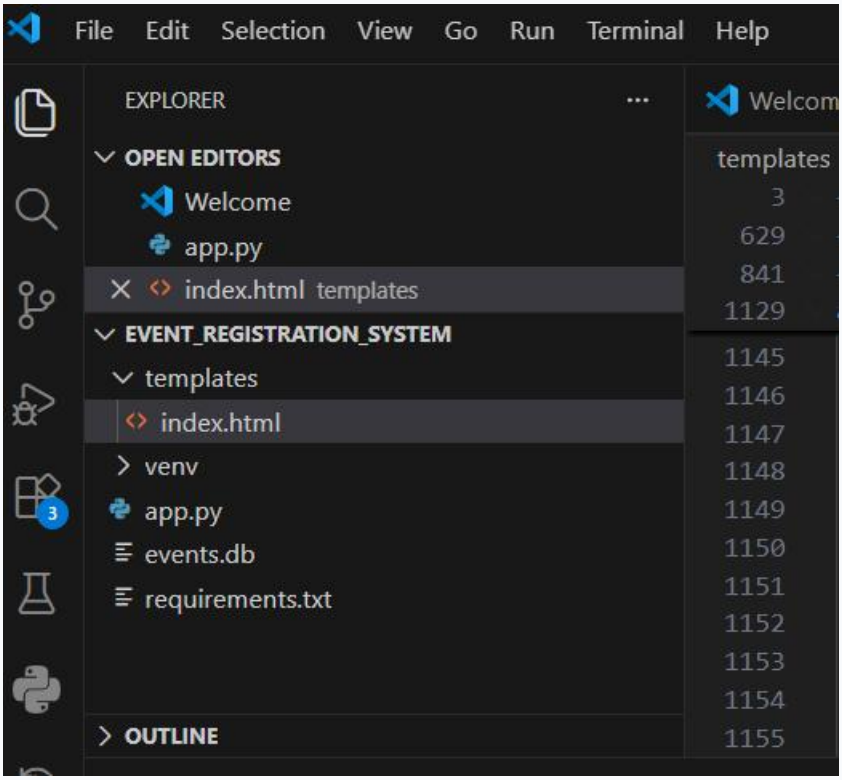
Perform the tests below on the running application and record the observed results. Mark a test as passed only after confirming it on your own system.

Test	Action	Expected Result
T1	Open the homepage	Dashboard loads without an error
T2	Create an event with valid data	Success message; event appears

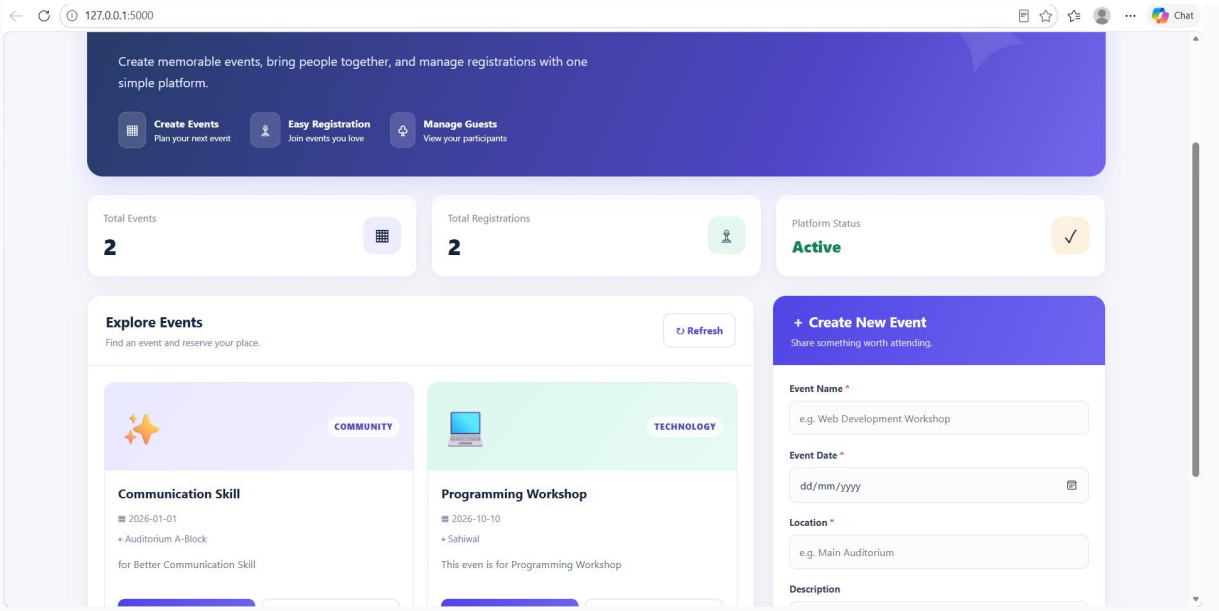
		in the list
T3	Refresh or restart the app	Saved event remains in the database
T4	Submit an event with a required field missing	Event is rejected with a validation message
T5	Register for an existing event	Registration success message is shown
T6	Open Guests for that event	Registered name and email are displayed
T7	Request registrations for an unknown event ID	Backend returns a 404 error

13. Screenshots and Evidence

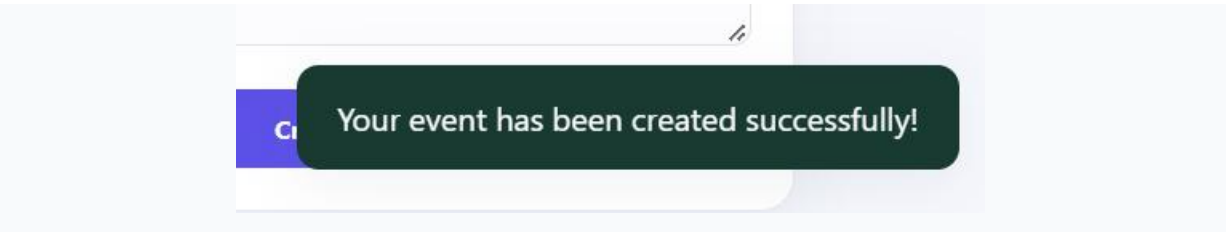
Screenshot 1: Project Structure in Visual Studio Code



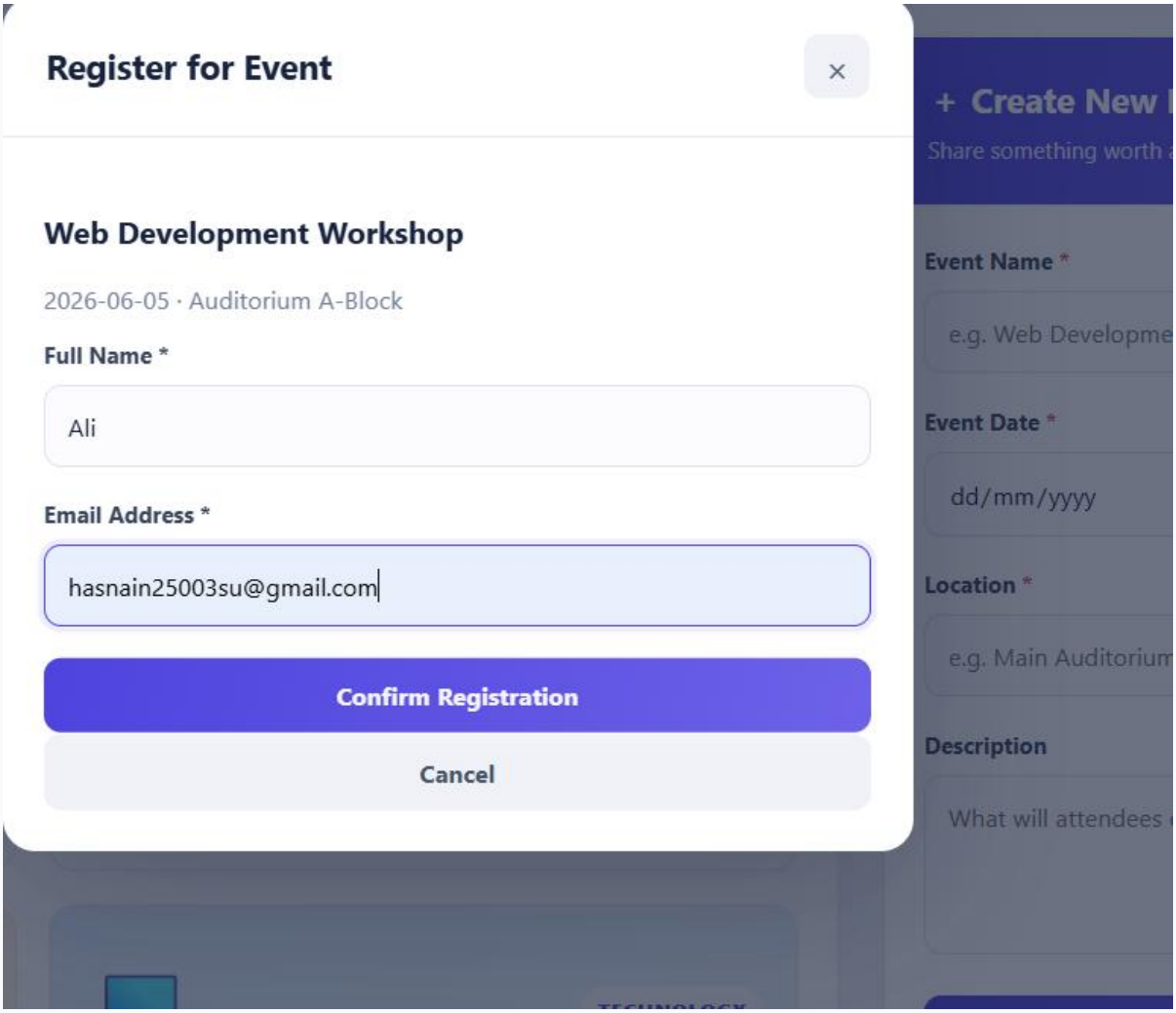
Screenshot 2: Professional Event Dashboard



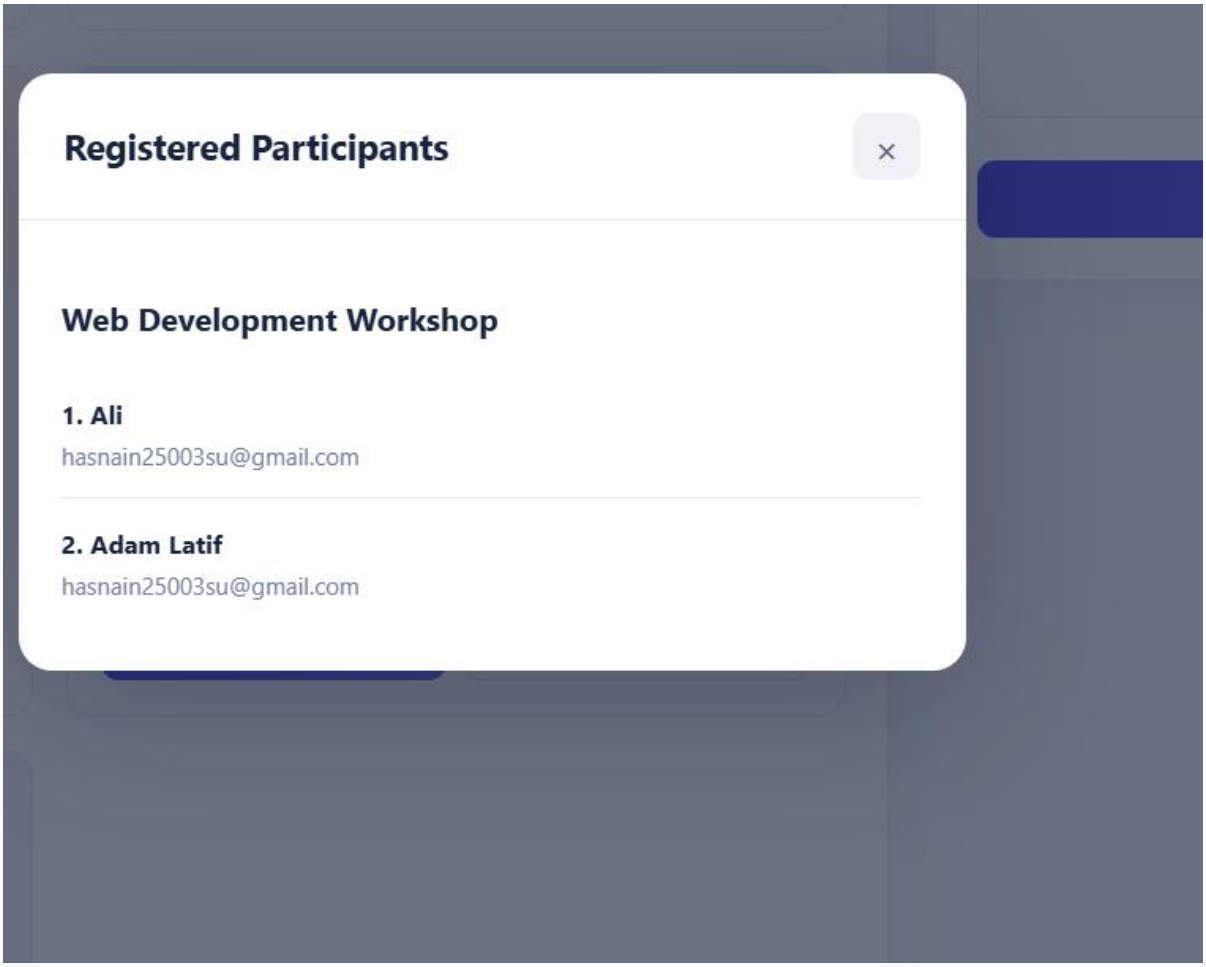
Screenshot 3: Successful Event Creation



Screenshot 4: Participant Registration



Screenshot 5: Registered Participants



14. Limitations and Future Improvements

Current limitations:

- The application runs locally and is not deployed to a public server.
- User accounts and administrator authentication are not included.
- Email addresses are not verified and duplicate registrations are allowed.
- There is no event capacity limit, cancellation feature, or email notification.
- The project needs additional security hardening before production use.

Possible future improvements:

- Add login and role-based access for organizers and participants.
- Prevent duplicate registrations for the same event.
- Add event editing, deletion, capacity limits, and cancellation.
- Add search, category filters, and event sorting.
- Deploy the application online with production security settings.

15. Conclusion

The Event Registration System demonstrates how a web interface can work with a Python Flask backend and an SQLite database. It supports creating events, listing saved events, registering participants, and viewing registrations. The project provides practice with HTTP methods, JSON, SQL queries, form validation, and responsive frontend design. It can be extended with authentication, event capacity, duplicate checks, and deployment features.

16. References

- Python Documentation — https://docs.python.org/3/
- Flask Documentation — https://flask.palletsprojects.com/
- SQLite Documentation — https://www.sqlite.org/docs.html
- MDN Web Docs — https://developer.mozilla.org/